

FOUNDATION_DESIGN.md

2026-04-16

準備基盤設計

目的

`qeyes` を継続的に実装・保守・リリースできる状態を整えるため、作業ルール、文書構成、リソース運用、版管理の基盤を定義する。

対象

- 実装方針
- 文書方針
- リソース方針
- README 方針
- リリース時の基礎運用
- テンプレート整備

対象外

- 個別機能の詳細仕様
- UI の細部デザイン
- アルゴリズムの最適化方針
- 各プラットフォーム固有の実装詳細

完了条件

次の条件を満たしたとき、この準備基盤は整ったものとみなす。

- `AGENTS.md` が存在する
- アプリ実装方針が定義されている
- リソース運用方針が定義されている
- 文書運用方針が定義されている
- ルート `README.md` が存在する
- `docs/README.md` が存在する
- `docs/history/RELEASE_HISTORY.md` が存在する
- `docs/guides/USER_GUIDE-template.md` が存在する
- `docs/releases/ReleaseNote-template.md` が存在する

設計方針

- ルールは `AGENTS.md` に集約する
- 利用者向けの入口はルート `README.md` に置く
- `docs/README.md` は `docs/` ディレクトリの案内に限定する
- 版付き文書は `docs/guides/` と `docs/releases/` に分離する
- 履歴は `docs/history/RELEASE_HISTORY.md` に集約する
- 過去版の完全な状態は Git タグで追跡する

将来拡張

次のような文書種別が増える場合は、この基盤設計の延長として扱う。

- `docs/architecture/`
- `docs/design/`

- docs/testing/

その場合は、 docs/README.md と AGENTS.md の両方を更新する。

DELIVERABLES.md

2026-04-16

成果物一覧

目的

準備基盤によって生成・維持される成果物を明確化する。

主要成果物

- AGENTS.md
 - 実装、文書、リソース、README の運用ルール
- README.md
 - プロジェクト全体の入口
- docs/README.md
 - docs/ ディレクトリの案内
- docs/history/RELEASE_HISTORY.md
 - 変更履歴
- docs/guides/USER_GUIDE-<version>.md
 - 版付き利用ガイド
- docs/releases/ReleaseNote-<version>.md
 - 版付きリリースノート

テンプレート

- docs/guides/USER_GUIDE-template.md
 - 利用ガイド作成用テンプレート
- docs/releases/ReleaseNote-template.md
 - リリースノート作成用テンプレート

補助文書

- docs/*.md
 - 運用メモ、判断記録、調査結果
- docs/foundation/*.md
 - この準備基盤自体の設計書

相互参照

- README.md は主要文書への導線を持つ
- docs/README.md は docs/ 配下の文書種別を示す
- AGENTS.md は各成果物の作成・更新ルールを定義する
- 版付き文書の追加時は docs/history/RELEASE_HISTORY.md を更新する

MAINTENANCE_RULES.md

2026-04-16

更新ルール

目的

文書やルールが増えたときに、どこを更新すべきかを判断できるようにする。

基本ルール

- 既存種別の文書を追加した場合は、必要に応じて `docs/history/RELEASE_HISTORY.md` を更新する
- 新しい文書種別を追加した場合は、`docs/README.md` にその種別と役割を追記する
- 利用者が参照すべき主要文書を追加した場合は、ルート `README.md` のリンクも更新する
- 文書配置や命名ルール自体に影響する場合は、`AGENTS.md` も更新する
- 文書を追加した後は `git status` を確認し、未追跡のまま残っていないことを確認する
- 他ホストで参照させる前に、必要な文書が `add・commit・push` 済みであることを確認する

工程ごとの修正ルール

- 各工程で問題が起きた場合は、まずその工程の文書を修正対象とする
- 修正対象には、その工程の入力文書、テンプレート、`AGENTS.md`、`GENERATION_INSTRUCTION.md` を含めて確認する
- `request` の問題は `REQUEST.md` と `docs/request/qeyes/AGENTS.md` を中心に見直す
- `requirements` の問題は要件定義書、要件テンプレート、`docs/requirements/qeyes/AGENTS.md` を中心に見直す
- `architecture` の問題は概要設計書、概要設計テンプレート、`docs/architecture/qeyes/AGENTS.md` を中心に見直す
- `design` の問題は詳細設計書、詳細設計テンプレート、`docs/design/qeyes/AGENTS.md` を中心に見直す
- `implementation` の問題は実装工程文書と `docs/implementation/qeyes/AGENTS.md` を中心に見直す
- `verification` の問題は検証工程文書と `docs/verification/qeyes/AGENTS.md` を中心に見直す
- 問題が上流工程に起因すると判断できる場合は、その上流工程まで戻って修正する

README の役割分担

- `README.md`
 - プロジェクト全体の説明
- `docs/README.md`
 - `docs/` ディレクトリの案内

判断表

- 既存種別の版付きガイドを追加する
 - `docs/guides/USER_GUIDE-<version>.md`
 - `docs/history/RELEASE_HISTORY.md`
 - 必要なら `README.md`
- 既存種別の版付きリリースノートを追加する
 - `docs/releases/ReleaseNote-<version>.md`
 - `docs/history/RELEASE_HISTORY.md`
 - 必要なら `README.md`
- 新しい文書種別を追加する
 - `docs/README.md`
 - `AGENTS.md`
 - 必要ならテンプレート
- ルール文書を追加する

- docs/README.md
 - AGENTS.md
- README の役割を変える
 - README.md
 - docs/README.md
 - AGENTS.md

非目標

- 個別機能の仕様判断
- 実装コードの設計判断
- UI デザインの最終判断

GENERATION_INSTRUCTION.md

2026-04-16

生成指示文

この文書の指示は、空の `qeyes/` ディレクトリを作成し、その直下に `AGENTS.md`、`FOUNDATION_DESIGN.md`、`DELIVERABLES.md`、`MAINTENANCE_RULES.md` を配置した状態を前提にする。

標準の生成指示文は次の通りです。

AGENTS.md に従って、
FOUNDATION_DESIGN.md、DELIVERABLES.md、MAINTENANCE_RULES.md を生成してください。

更新時は次のように指示します。

AGENTS.md の内容に合わせて
FOUNDATION_DESIGN.md、DELIVERABLES.md、MAINTENANCE_RULES.md を更新してください。

SETUP_DESIGN_NOTE.md

2026-04-16

準備内容を生成するための設計書について

設計書は「アプリそのものの設計」ではなく、「アプリを作れる状態をどう整えるかの設計」に分けるのがよいです。つまり実装対象は `qeyes` ではなく、`qeyes` を継続的に作れる開発基盤です。

おすすめは 1 枚に全部詰めず、次の 3 層で分ける形です。

1. 目的と範囲の設計書

名前の例:

- `docs/foundation/FOUNDATION_DESIGN.md`

書く内容:

- 何の準備を対象にするか
- どこまでを「準備完了」とみなすか
- 何をこの設計書の対象外にするか

例:

- 実装方針
- 文書方針
- リソース方針
- リリース方針
- テンプレート
- ルール文書
- 対象外
- 実際の機能設計詳細
- UI の細部仕様

2. 成果物一覧の設計書

名前の例:

- `docs/foundation/DELIVERABLES.md`

書く内容:

- 準備によって何が生成・整備されるか
- 各ファイルの役割
- 相互参照

例:

- `AGENTS.md`
- `README.md`
- `docs/README.md`
- `docs/history/RELEASE_HISTORY.md`
- `docs/guides/USER_GUIDE-<version>.md`
- `docs/releases/ReleaseNote-<version>.md`
- テンプレート類

これがあると、「準備の完了条件」が見えやすいです。

3. 更新ルール設計書

名前の例:

- docs/foundation/MAINTENANCE_RULES.md

書く内容:

- 新バージョン時に何を更新するか
- 新文書種別追加時に何を更新するか
- README と docs README の役割分担
- ルール変更時の反映先

これは今 AGENTS.md に一部入っていますが、設計書としては別にまとめた方が読みやすいです。

推奨構成

```
docs/  
  foundation/  
    FOUNDATION_DESIGN.md  
    DELIVERABLES.md  
    MAINTENANCE_RULES.md
```

設計書に最低限入れるべき観点

- 目的
- 完了条件
- 成果物一覧
- 命名規則
- 更新規則
- 役割分担
- 例外処理
- 将来拡張方針

特に重要なのは「完了条件」です。

今回で言うと、例えば次のように書けます。

- AGENTS.md が存在する
- 実装方針、文書方針、リソース方針が定義されている
- ルート README.md が存在する
- docs/README.md が存在する
- 版付き利用ガイドと版付きリリースノートのテンプレートが存在する
- リリース履歴ファイルが存在する

この形にすると、設計書が単なる説明ではなく、チェック可能な仕様になります。

実務上さらに重要な点

- 判断表
 - 何が増えたときに何を更新するか
- 非目標
 - 何はこの基盤設計の責任外か

例えば判断表です。

- 既存種別の文書追加
 - `RELEASE_HISTORY.md`
- 新文書種別追加
 - `docs/README.md` , `AGENTS.md`
- 利用者向け主要文書追加
 - `README.md`
- 命名規則変更
 - `AGENTS.md` , テンプレート

結論

設計書は次の方針で作るのがよいです。

- 1枚に全部書かない
- 「目的」「成果物」「更新規則」を分ける
- 完了条件を明文化する
- 判断表を入れる

qeyes/AGENTS.md

2026-04-16

AGENTS

アプリ実装方針

`qeyes` は、マウスポインターを追う目を常時表示する小さな Qt Widgets アプリとして実装する。

基本方針

- `Qt6::Widgets` を使う。
- 描画は `QPainter` で行う。
- QML / Qt Quick は使わない。
- 軽量な常駐ツールとして振る舞うことを優先する。
- 不要に大きな依存や複雑な構成を追加しない。

UI 方針

- ウィンドウはフレームレスとする。
- 背景は透過にする。
- 最前面表示を維持する。
- 左ドラッグで移動できるようにする。
- 左ダブルクリックで終了できるようにする。
- 設定 UI は右クリックから開けるようにする。

描画方針

- 目はコードで動的に描画する。
- 画像アセットに依存しない。
- 目の構成要素は輪郭、白目、虹彩、瞳孔とする。
- 視線はマウスポインター方向へ追従する。
- 追従時も各要素が白目の外にはみ出さないようにする。

設定方針

- 永続化には `QSettings` を使う。
- 保存対象は明示された設定項目に限る。
- 保存キーは `colors/...` のように用途ごとに整理する。
- 一時的な見た目調整のために過剰な設定項目を増やさない。

プラットフォーム方針

- まず共通実装を優先する。
- macOS 固有の見た目差分は必要最小限に留める。
- 他プラットフォームに不要な特例は持ち込まない。

変更時の注意

- 既存操作を壊す変更は避ける。
- 見た目の変更は小さなツールとしての一貫性を保つ。
- 新機能追加時は、操作方法、保存仕様、文書も合わせて更新する。

リソース運用方針

アイコン

- アイコン関連ファイルは `assets/` 配下で管理する。

- 正本画像は `assets/icon-master.png` とし、各形式はこれを元に展開する。
- ファイル名は既存の命名を維持し、カレントディレクトリー名や別名を付けない。
- アプリ用アイコンを変更するときは、少なくとも次を整合させる。
- `assets/qeyes.icns`
- `assets/qeyes.ico`
- `assets/icon.iconset/icon_16x16.png`
- `assets/icon.iconset/icon_16x16@2x.png`
- `assets/icon.iconset/icon_32x32.png`
- `assets/icon.iconset/icon_32x32@2x.png`
- `assets/icon.iconset/icon_128x128.png`
- `assets/icon.iconset/icon_128x128@2x.png`
- `assets/icon.iconset/icon_256x256.png`
- `assets/icon.iconset/icon_256x256@2x.png`
- `assets/icon.iconset/icon_512x512.png`
- `assets/icon.iconset/icon_512x512@2x.png`
- `CMakeLists.txt` や `qeyes.rc` から参照されるファイル名は原則変更しない。

変更時の注意

- リソース差し替え時は、正本が `assets/icon-master.png` であることを前提に更新する。
- 生成物を更新した場合は、必要な派生ファイルもまとめて更新する。
- 一部プラットフォームだけ古いアイコンが残る状態を避ける。

文書運用ルール

このプロジェクトの文書は `docs/` 配下に次の構成で配置する。

```
docs/  
  *.md  
  guides/  
    USER_GUIDE-template.md  
    USER_GUIDE-<version>.md  
  releases/  
    ReleaseNote-template.md  
    ReleaseNote-<version>.md  
  history/  
    RELEASE_HISTORY.md
```

ルール

- 利用ガイドは `docs/guides/USER_GUIDE-<version>.md` に置く。
- リリースノートは `docs/releases/ReleaseNote-<version>.md` に置く。
- 全体の変更履歴は `docs/history/RELEASE_HISTORY.md` にまとめる。
- テンプレートは `docs/guides/USER_GUIDE-template.md` と `docs/releases/ReleaseNote-template.md` に置く。
- `docs/` 直下の補助文書は、運用メモや判断材料の記録として置いてよい。
- ドキュメントのファイル名は固定命名とし、カレントディレクトリー名やプロジェクト名を接頭辞・接尾辞として付けない。
- `USER_GUIDE.md`、`ReleaseNote.md`、`ReleaseNote.txt` のような重複ファイルは新たに作らない。
- 新しいバージョンを追加するときは、次をまとめて更新する。
- ビルド定義上のバージョン番号
- `docs/guides/USER_GUIDE-<version>.md`
- `docs/releases/ReleaseNote-<version>.md`
- `docs/history/RELEASE_HISTORY.md`

- テンプレートの更新が必要ならそれも含める
- 同じ内容を `.md` と `.txt` の両方で管理しない。
- 過去バージョンの状態は Git タグで追跡する。
- Finder などが生成する `.DS_Store` のような不要ファイルは追跡しない。
- 文書を追加した後は `git status` を確認し、未追跡のまま残っていないことを確認する。
- 他ホストで参照させる前に、必要な文書が `add・commit・push` 済みであることを確認する。

文書追加時の更新ルール

- 既存種別の文書を追加した場合は、必要に応じて `docs/history/RELEASE_HISTORY.md` を更新する。
- 新しい文書種別を追加した場合は、`docs/README.md` にその種別と役割を追記する。
- 利用者が参照すべき主要文書を追加した場合は、ルート `README.md` のリンクも更新する。
- 文書配置や命名ルール自体に影響する場合は、`AGENTS.md` も更新する。

工程ごとの修正ルール

- 各工程で問題が起きた場合は、まずその工程の文書を修正対象とする。
- 修正対象には、その工程の入力文書、テンプレート、`AGENTS.md`、`GENERATION_INSTRUCTION.md` を含めて確認する。
- `request` の問題は `REQUEST.md` と `docs/request/qeyes/AGENTS.md` を中心に見直す。
- `requirements` の問題は要件定義書、要件テンプレート、`docs/requirements/qeyes/AGENTS.md` を中心に見直す。
- `architecture` の問題は概要設計書、概要設計テンプレート、`docs/architecture/qeyes/AGENTS.md` を中心に見直す。
- `design` の問題は詳細設計書、詳細設計テンプレート、`docs/design/qeyes/AGENTS.md` を中心に見直す。
- `implementation` の問題は実装工程文書と `docs/implementation/qeyes/AGENTS.md` を中心に見直す。
- `verification` の問題は検証工程文書と `docs/verification/qeyes/AGENTS.md` を中心に見直す。
- 問題が上流工程に起因すると判断できる場合は、その上流工程まで戻って修正する。

README 運用ルール

- ルートの `README.md` が存在する場合は、このリポジトリの入口として扱う。
- `README.md` を作成または更新する場合は、少なくとも次を含める。
- プロジェクト概要
- ビルド方法
- 起動方法
- 主な操作
- 関連文書へのリンク
- `docs/README.md` は `docs/` 配下の案内として扱い、詳細な運用ルールは必要に応じて別文書へ分ける。
- `README.md` には詳細な文書運用ルールを重複して長く書きすぎない。
- 新機能追加やバージョン更新で利用者向けの説明が変わる場合は、`README.md` も更新対象に含める。